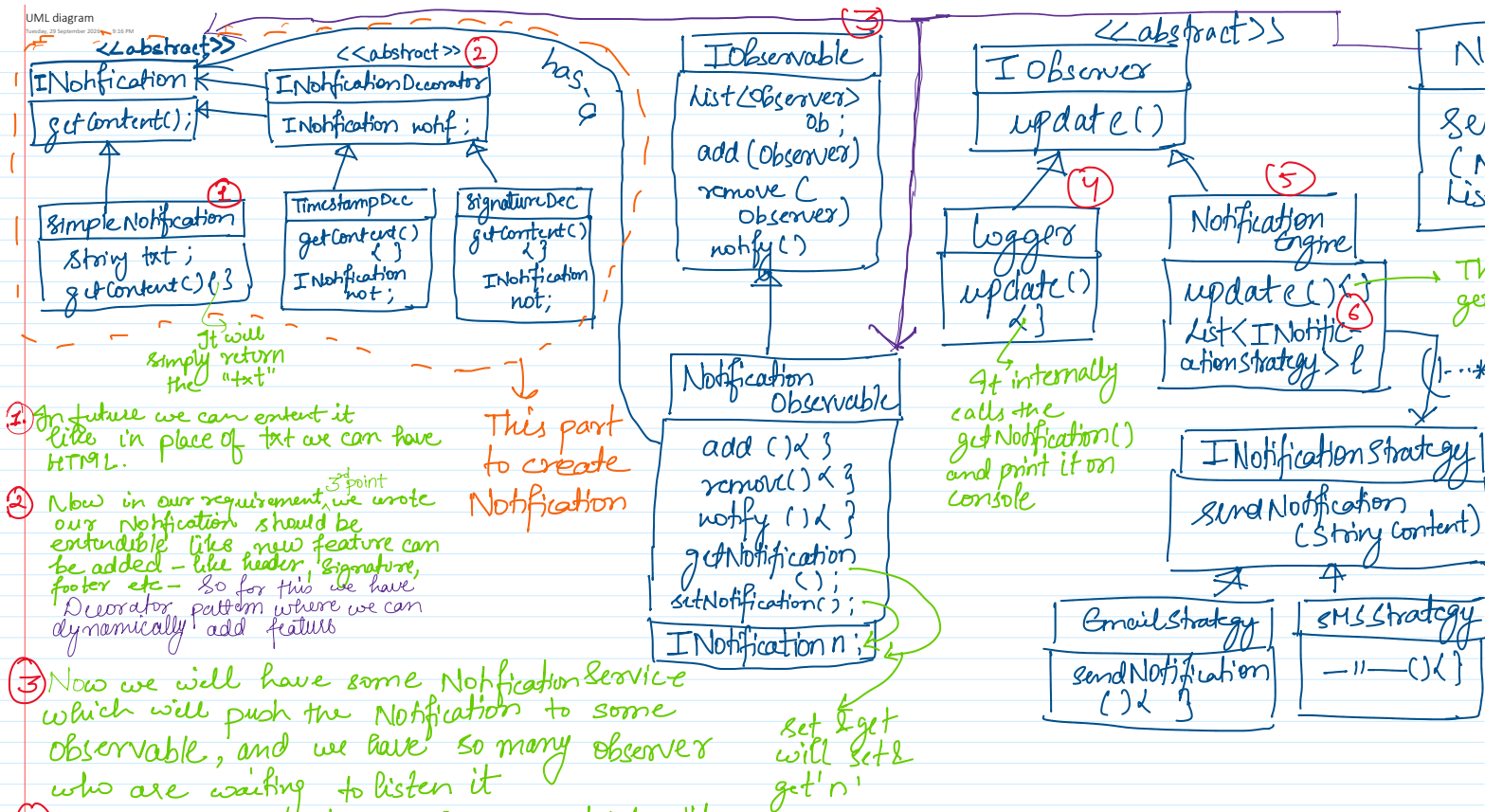


Design a Notification System

Requirements:-

- ① Plug & Play Model
— Easy to integrate with any application
- ② Extendible : SMS, Email, pop-up
— In future if we want we can add whatsapp etc.
- ③ Notification System extendible means extra feature addition should be easy.
- ④ Store Notification in DB
- ⑤ Logging the notification. for now we will log it on console.



1. In future we can extent it like in place of txt we can have HTML.

2. Now in our requirement, we wrote our Notification should be extendible like new feature can be added - like header, signature, footer etc - So for this we have Decorator pattern where we can dynamically add features

3. Now we will have some NotificationService which will push the Notification to some observable, and we have so many observer who are waiting to listen it

4. log is also a type of observer which will log the Notification, whenever it comes on console

5. Notification Engine will transfer the Notification over the Internet. we will make it abstract so that in future we can extent it like by adding WhatsApp. NotificationEngine don't have knowledge by what means like Email, WhatsApp, SMS we are sending Notification - So here we use strategy Pattern

6. Generally in Strategy Design Pattern Client has only 1 instance of INotificationStrategy but here we want to send Notification on SMS, Email everywhere. That's why we have take multiple instance of INotificationStrategy.

7. Now what we need is NotificationService which we create the 'Notification' & give it to observable to send it via SMS, Email or print on console

8. It will have method sendNotification() which will take (Notification n) created by user and assign to observable
 ∴ It should have reference of Observable.
 It is also dealing with Notification
 ∴ It should have reference of Notification as well

9. We also wanted to store the Notification.

```

NotificationService {
  <<Singleton>>
  sendNotification(Notification n) { }
  + <Notification> list
}

```

This will also call
 sendNotification() and
 loop through 'l'
 and call
 sendNotification()
 for each in 'l'.

```

class PopUpStrategy {
  -u-() { }
}

```

∴ it should have reference of Notification in it

(9) We also wanted to store the Notification for that we can take a list <Notification> list → sendNotification() will also add the Notification in this list.

(10) We want only one history of Notification so our NotificationService will be singleton.

Next & Clean UML Diagram → see Notes.

Code → see Notes.

